



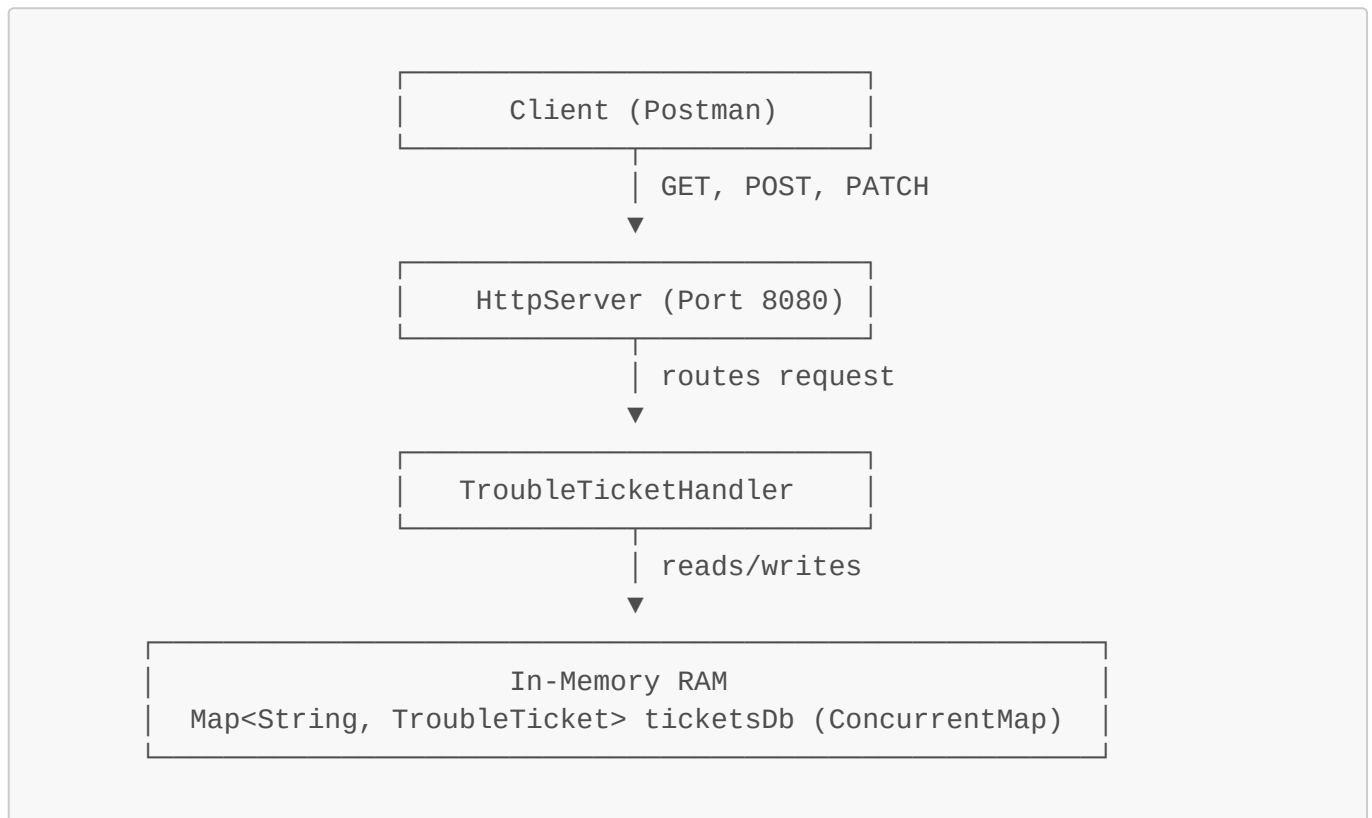
Trouble Ticket API Code Walkthrough

This document explains how the pure Java implementation of the **TMF621 Trouble Ticket API** works.

Rather than using complex frameworks like Spring Boot, this project uses standard Java libraries (like `com.sun.net.httpserver.HttpServer`) and a single JSON processing library (**Jackson**) to handle REST operations.



High-Level Architecture



1. **HttpServer**: Listens on port **8080** and receives incoming HTTP requests.
 2. **TroubleTicketHandler**: Routes requests based on their HTTP method (**GET**, **POST**, **PATCH**) and parses URLs.
 3. **In-Memory Database**: Stored in a thread-safe Java `ConcurrentHashMap` in RAM. No physical database files are created.
-

1. The Data Models (`com.example.troubleticket.model`)

These are **POJOs** (Plain Old Java Objects) that represent the resources defined by the TMF621 standard.

Conforming to JSON Standards (`@JsonProperty`)

To match the TMF specifications, some JSON keys must start with `@` (e.g., `@type`, `@baseType`, `@schemaLocation`, `@referredType`). Java variables cannot start with `@`, so we use Jackson's `@JsonProperty` annotation:

```
@JsonProperty("@baseType")
private String baseType;

@JsonProperty("@type")
private String type;
```

When Jackson converts the Java object into JSON (or vice-versa), it translates the variable name `baseType` to `@baseType`.

TroubleTicket Class (`TroubleTicket.java`)

This is the root model. It holds basic attributes as well as nested lists of sub-resources:

```
public class TroubleTicket {
    private String id;
    private String href;
    private String name;
    private String description;
    private String severity;
    private String status;

    // Lists of nested sub-resources
    private List<Note> note = new ArrayList<>();
    private List<StatusChange> statusChange = new ArrayList<>();
    private List<Attachment> attachment = new ArrayList<>();
    private List<RelatedParty> relatedParty = new ArrayList<>();
}
```

Note: All dates (like `creationDate`, `lastUpdate`) are represented as simple `Strings` to avoid timezone parsing bugs and keep the code simple for beginners.

2. The Server & Routing (`TroubleTicketServer.java`)

This is the entry point and core logic of the application. It contains two main parts:

1. The `main` method to start the server.
2. The `TroubleTicketHandler` to process the requests.

A. Starting the Server

```
public static void main(String[] args) throws IOException {
    int port = 8080;
    // Create an HTTP Server listening on port 8080
    HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);

    // Route all paths starting with /troubleTicket/v2/troubleTicket to our
    handler
```

```
server.createContext("/troubleTicket/v2/troubleTicket", new
TroubleTicketHandler());

server.start();
System.out.println("Server started successfully!");
}
```

B. Routing Incoming Requests (`handle` method)

The handler determines the HTTP method and calls the corresponding helper method:

```
@Override
public void handle(HttpExchange exchange) throws IOException {
    String method = exchange.getRequestMethod();
    String path = exchange.getRequestURI().getPath();

    // Set response headers to JSON
    exchange.getResponseHeaders().set("Content-Type", "application/json");

    if ("GET".equalsIgnoreCase(method)) {
        handleGet(exchange, path);
    } else if ("POST".equalsIgnoreCase(method)) {
        handlePost(exchange);
    } else if ("PATCH".equalsIgnoreCase(method)) {
        handlePatch(exchange, path);
    } else {
        sendResponse(exchange, 405, new ErrorResponse("405", "Method Not
Allowed", "HTTP Method not supported"));
    }
}
```

3. GET Endpoint (`handleGet`)

Handles two scenarios:

1. **Retrieve by ID:** `GET /troubleTicket/v2/troubleTicket/{id}`
2. **List all / Filter:** `GET /troubleTicket/v2/troubleTicket?status=Acknowledged`

```
private void handleGet(HttpExchange exchange, String path) throws
IOException {
    // 1. Retrieve a single ticket by ID
    if (path.matches("^/troubleTicket/v2/troubleTicket/[^/]+$")) {
        String id = path.substring(path.lastIndexOf('/') + 1);
        TroubleTicket ticket = ticketsDb.get(id);

        if (ticket == null) {
            sendResponse(exchange, 404, new ErrorResponse("404", "Not
Found", "TroubleTicket with id '" + id + "' does not exist"));
        }
    }
}
```

```
        } else {
            sendResponse(exchange, 200, ticket);
        }
    }
    // 2. List all tickets (supports query filtering)
    else if (path.equals("/troubleTicket/v2/troubleTicket")) {
        Map<String, String> filters =
        parseQueryParams(exchange.getRequestURI().getQuery());
        List<TroubleTicket> list = new ArrayList<>(ticketsDb.values());

        if (filters.containsKey("status")) {
            list = list.stream().filter(t ->
            filters.get("status").equalsIgnoreCase(t.getStatus())).toList();
        }
        sendResponse(exchange, 200, list);
    }
}
```

4. POST Endpoint (`handlePost`)

Handles creation of new tickets with validation:

```
private void handlePost(HttpExchange exchange) throws IOException {
    // Deserialize request body JSON into a Java TroubleTicket object
    TroubleTicket ticket =
    objectMapper.readValue(exchange.getRequestBody(), TroubleTicket.class);

    // 1. Check mandatory fields
    List<String> missingFields = new ArrayList<>();
    if (isEmpty(ticket.getDescription())) missingFields.add("description");
    if (isEmpty(ticket.getSeverity())) missingFields.add("severity");
    if (isEmpty(ticket.getTicketType())) missingFields.add("ticketType");

    if (!missingFields.isEmpty()) {
        sendResponse(exchange, 400, new ErrorResponse("400", "Bad Request",
        "Mandatory fields [" + String.join(", ", missingFields) + "] are
        missing"));
        return;
    }

    // 2. Generate values
    if (ticket.getId() == null) ticket.setId(UUID.randomUUID().toString());
    ticket.setHref("http://localhost:8080/troubleTicket/v2/troubleTicket/"
    + ticket.getId());

    // Set default status
    if (ticket.getStatus() == null) ticket.setStatus("Acknowledged");

    // 3. Create first audit history log entry
    if (ticket.getStatusChange().isEmpty()) {
```

```
        ticket.getStatusChange().add(new StatusChange(ticket.getStatus(),
"Ticket created", "TroubleTicket"));
    }

    // 4. Save in-memory Map
    ticketsDb.put(ticket.getId(), ticket);
    sendResponse(exchange, 201, ticket);
}
```

5. PATCH Endpoint (`handlePatch`)

Handles partial updates (merge patch semantics). Only updates the fields supplied by the client, and automatically handles state machine audit log creation.

```
private void handlePatch(HttpExchange exchange, String path) throws
IOException {
    String id = path.substring(path.lastIndexOf('/') + 1);
    TroubleTicket ticket = ticketsDb.get(id);

    if (ticket == null) {
        sendResponse(exchange, 404, new ErrorResponse("404", "Not Found",
"TroubleTicket does not exist"));
        return;
    }

    // Read request body as a flexible Map of key-value pairs
    Map<String, Object> updates =
objectMapper.readValue(exchange.getRequestBody(), new
TypeReference<Map<String, Object>>(){});

    String originalStatus = ticket.getStatus();
    boolean statusChanged = false;
    String newStatus = null;
    String statusChangeReasonText = null;

    for (Map.Entry<String, Object> entry : updates.entrySet()) {
        String key = entry.getKey();
        Object value = entry.getValue();

        // SILENTLY IGNORE read-only fields
        if (key.equals("id") || key.equals("href") ||
key.equals("creationDate") || key.equals("lastUpdate") ||
key.equals("statusChange")) {
            continue;
        }

        switch (key) {
            case "description":
                ticket.setDescription((String) value);
                break;

```

```

        case "status":
            newStatus = (String) value;
            if (newStatus != null &&
!newStatus.equalsIgnoreCase(originalStatus)) {
                statusChanged = true;
            }
            break;
        case "statusChangeReason":
            statusChangeReasonText = (String) value;
            ticket.setStatusChangeReason(statusChangeReasonText);
            break;
        case "note":
            // Append new notes to existing list
            List<Note> newNotes = objectMapper.convertValue(value, new
TypeReference<List<Note>>() {});
            ticket.getNote().addAll(newNotes);
            break;
        // Additional fields handled in actual code...
    }
}

// If status changed, generate a new history audit entry
if (statusChanged && newStatus != null) {
    ticket.setStatus(newStatus);
    String reason = statusChangeReasonText != null ?
statusChangeReasonText : "Status updated via PATCH";
    ticket.getStatusChange().add(new StatusChange(newStatus, reason,
"TroubleTicket"));
}

ticket.setLastUpdate(new SimpleDateFormat("yyyy-MM-
dd'T'HH:mm:ss").format(new Date()));
ticketsDb.put(ticket.getId(), ticket);
sendResponse(exchange, 200, ticket);
}

```

6. How Integration Tests Work (TroubleTicketApplicationTests.java)

We write automated integration tests to make sure that our server logic behaves correctly under all conditions.

- **Background Server Start:** Using `@BeforeAll`, the test framework starts the HTTP server on a non-conflicting port (8081).
- **HTTP Client:** Uses Java's native `HttpClient` to send requests:

```

HttpRequest request = HttpRequest.newBuilder()

    .uri(URI.create("http://localhost:8081/troubleTicket/v2/troubleTicket"
))

```

```
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(payload))
        .build();
    HttpResponse<String> response = client.send(request,
    HttpResponse.BodyHandlers.ofString());
```

- **Assertions:** Compares the HTTP response status code and body properties with expected outcomes (e.g., `assertEquals(201, response.statusCode())`).
- **Background Server Shutdown:** Using `@AfterAll`, it safely stops the test server.